



Assignment

Unit 5: Practice & Assessment Questions

Q01

Assembly vs. HLL

Explain structural and performance differences between assembly language and high-level languages.

Q02

Assembler Passes

Differentiate between 1-pass and 2-pass assemblers. Why is a second pass necessary?

Q03

Addressing Modes

Illustrate Register, Direct, and Indirect addressing modes with assembly code snippets.

Q04

Instruction Cycle

Trace the exact sub-phases of Fetch, Decode, and Execute for a standard MOV instruction.

Q05

Pipelining Basics

Explain how overlapping instruction execution stages enhances modern CPU throughput.

Q06

Pipeline Hazards

Identify and define Structural, Data, and Control hazards in a instruction pipeline.

Q07

Hazard Mitigation

Describe solutions to pipeline stalls, including operand forwarding and branch prediction.

Q08

Vector Processing

Compare SIMD architecture with scalar execution. How does it optimize data-heavy tasks?

Q09

RISC vs. CISC

Analyze instruction set differences and pipeline compatibility between RISC and CISC.

Q10

Performance Math

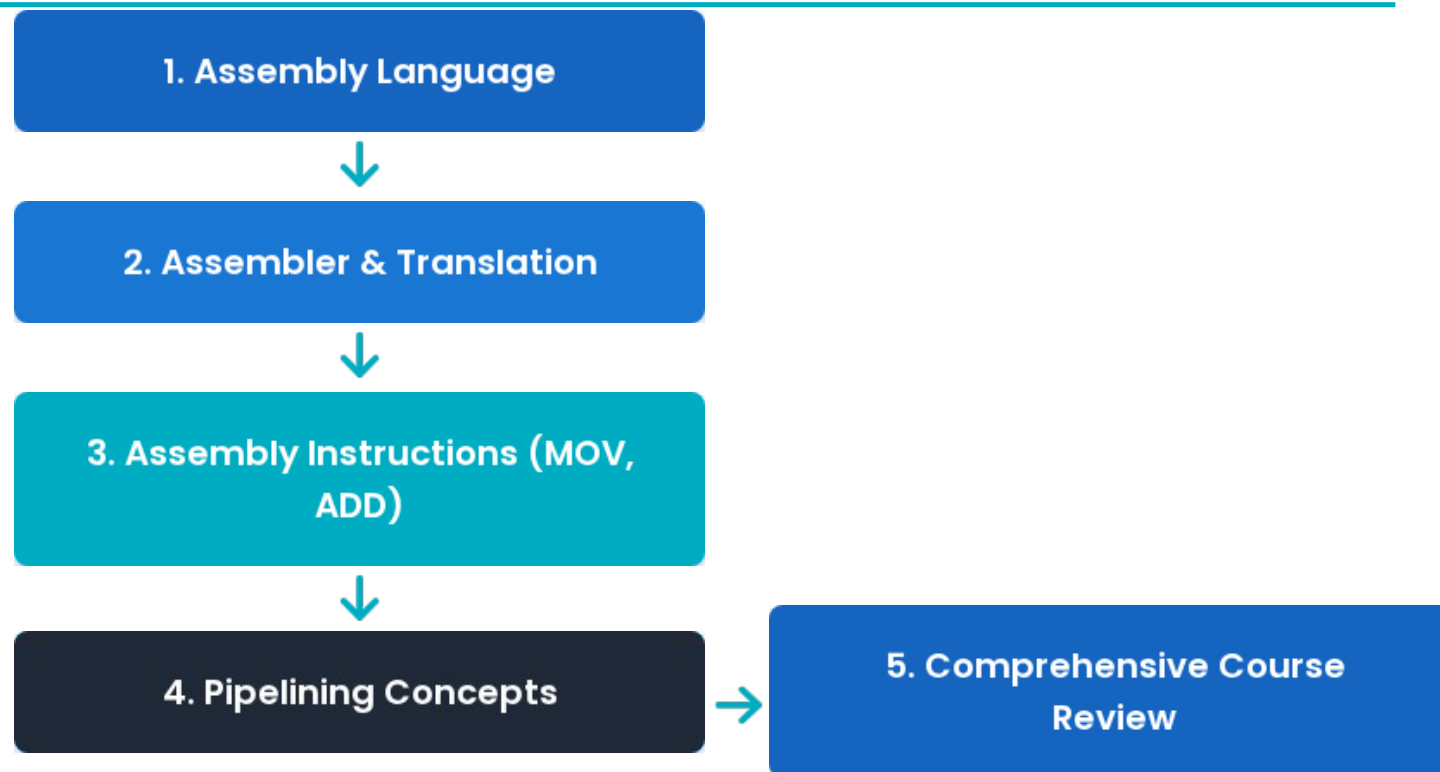
Calculate speedup factor of an n-stage pipeline over non-pipelined system for N tasks.

Course Outcome 5



Overview

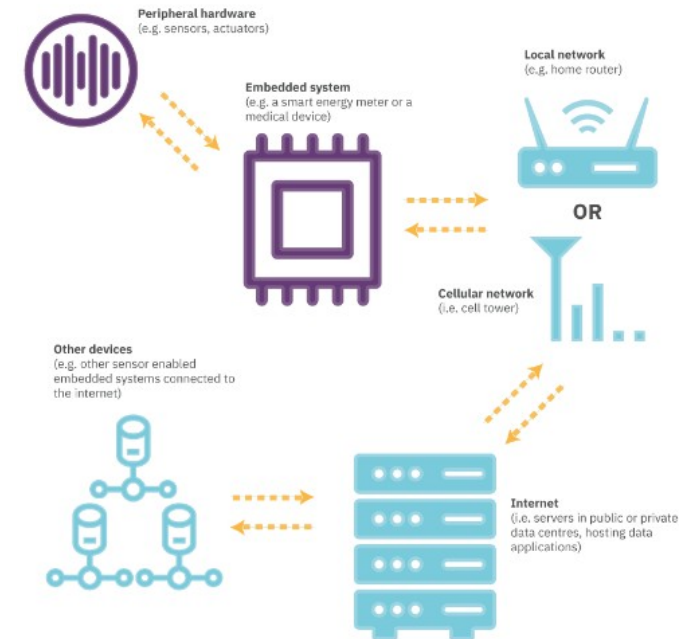
Visual Roadmap of Unit 5



Where is this used in the real world?

- **Embedded Systems:** Microcontrollers in home appliances require tight, efficient
- **Operating Systems:** Bootloaders and assembly code.
- kernel-level hardware management use
- **Device Drivers:** Direct communication with Assembly.
- hardware peripherals requires low-level
- **Robotics & Automotive:** ECUs in modern instructions.
- vehicles demand real-time execution speeds.
- **IoT Devices:** Resource-constrained devices
- rely on optimized assembly for power
- **Cyber Security:** Reverse engineering and efficiency.
- malware analysis heavily rely on reading assembly.

IoT and embedded systems



Primary Goals of this Course



Understand Architecture

To conceptualize the basic structure and operation of a digital computer system.



Low-Level Programming

To master the fundamental concepts of assembly language programming and instruction execution.



Performance Enhancement

To analyze pipelining techniques and how they improve modern CPU throughput and performance.



Outcomes (COs)

What you will achieve



Outcomes (POs)

Engineering Graduate Attributes



Mapping

Alignment Matrix



Outcomes (PSOs)



Mapping



Objectives

-
-
-



Dashboard



Question Paper



Prerequisites

-
-
-
-
-



Bridging the Gap

Assembly Language is a low-level

programming language that uses mnemonics to

- **Machine Dependent:** Specific to a particular represent machine-level instructions.

- computer architecture (e.g., x86, ARM, MIPS).

- **Human Readable:** Easier to read than raw

- **Direct (Hardware) Control:** Allows manipulation of specific registers and memory addresses.

```
; Example x86 Assembly
MOV AX, 0005H ; Load 5 into AX
MOV BX, 0003H ; Load 3 into BX
ADD AX, BX ; Add BX to AX (AX=8)
HLT ; Halt execution
```



Recommended Study Materials

Textbooks:

- "Course Outcome 5 and Design" by Patterson & Hennessy
- "Computer System Architecture" by M. Morris Mano
- "Computer Architecture: A Quantitative Approach"

Digital Resources:

- NPTEL Video Lectures on Computer Architecture
- Online Simulators (e.g., MARS for MIPS, CPU-OS Simulator)



Roadmap

Journey through the Processor



Our goal is to understand how code reaches the hardware and runs efficiently.



Objectives

Specific Goals for Unit 5

- Understand the syntax and semantics of Assembly Language.
- Comprehend the internal working of an Assembler.
- Write basic assembly programs using standard instructions.
- Differentiate clearly between Assembly and Machine code.
- Master the concept of CPU Pipelining and its performance impact.



Assembly Language

Objective & Introduction

Topic Objective: To introduce the concept of low-level programming and its significance in

Topic Outcome: Students will be able to define assembly language and explain its need over high-level and machine languages.

Introduction

Before compilers were highly optimized, programmers needed a way to write fast, hardware-specific code without typing raw binary. Assembly language provides symbolic names (mnemonics) for the numeric machine codes that the CPU understands.

Definition & Detailed Theory

Definition

Assembly language is a low-level programming language that has a strong correspondence between its instructions and the architecture's machine code instructions.

Detailed Theory

- Every CPU family (x86, ARM, RISC-V) has its own distinct **assembly mnemonics** (like ADD, SUB, JMP) instead of binary codes.
- It exposes the programmer directly to the CPU's registers, memory architecture, and flags.
- Requires an **Assembler** utility to translate it into machine code.

```
L, '-' ; get '-'  
H, 2 ; print char function  
1H ; print '-'  
AX ; get AX back  
AX ; AX = -AX  
  
; decimal digits  
CX, 0 ; CX counts digits  
BX, 10D ; BX has divisor  
  
; prepare high word of  
DX, 0 ; AX = quotient, DX = remainder  
BX ; save remainder as high word  
DX ; count = count + 1
```



Assembly Language

Need & Characteristics

Why do we need it?

- Absolute control over hardware
- Critical performance optimization (game engines, OS kernels).
- Memory-constrained environments
- Accessing specialized processor instructions not exposed by high-level languages.

Key Characteristics

1-to-1 mapping with machine

Hardware-dependent (Non-portable).

Requires intimate knowledge of

Uses Symbolic Addressing.

Advantages & Disadvantages

👍 Advantages

- Highly efficient execution speed.
- Minimal memory footprint.
- Hardware-level diagnostics and debugging.
- No compiler overhead.
- Precise timing control (crucial for real-time systems).

👎 Disadvantages

- Very difficult and time-consuming to write.
- Code is not portable across different CPU architectures.
- Prone to human error; lacks high-level abstractions (like objects or easy memory management).
- High learning curve.



Assembly Language

Applications & Real-Life Analogy

Real-Life Analogy

Think of programming languages like giving directions to a driver.

High-Level (Python): "Drive to the supermarket." (The driver handles the route, gears, steering).

Assembly Level: "Depress clutch, engage 1st gear, press accelerator 10%, turn steering wheel 15 degrees right..." (You control the mechanical components directly).

Major Applications

- Writing bootloaders (BIOS/UEFI).
- Developing real-time embedded systems (Pacemakers, ABS brakes).
- Reverse engineering malware to understand its mechanism.



Assembly Language

Technical Diagram & Examples



```
; C++ Code: a = b + c;

; Assembly Equivalent (MIPS):
lw $t0, b ; Load b into register $t0
lw $t1, c ; Load c into register $t1
add $t2, $t0, $t1 ; Add $t0 and $t1, store in $t2
sw $t2, a ; Store $t2 into memory location a
```



Assessment & Resources

Daily Quiz

1. Why is assembly language considered "machine-dependent"?
2. Give one scenario where writing code in assembly is preferred over Python.
3. What are the symbolic names in assembly called? (Hint: Starts with 'M')

Topic Learning Link: https://www.youtube.com/watch?v=_BpOV3G1-m0&list=PLAZj-jE2aeZi0N7HdEgMpeD0p2ojmAcY

Important Exam Point: Always mention "Hardwired Dependency" when comparing Assembly to High-Level Languages.



Assembler

Objective & Definition

Topic Objective: To understand the software tool responsible for translating assembly code.

Topic Outcome: Students will be able to explain the working phases of a Two-Pass Assembler.

Definition

An **Assembler** is a system software program that translates assembly language code (source code) into machine language (object code) that the computer's CPU can directly execute.



Assembler

Role & File Types

The Core Role

The CPU cannot read "MOV A, B". It only understands binary patterns like "10110000". The assembler acts as the translator bridging this gap.

1. Source Program

The text file written by the programmer containing mnemonics and macros. (e.g., .asm file)

2. Object Program

The intermediate binary machine code output by the assembler, not yet linked. (e.g., .obj file)

3. Executable

The final ready-to-run file after the linker joins object files. (e.g., .exe or .elf file)

Working Process: Assembler Passes

One-Pass Assembler

Reads the source code once and generates machine code simultaneously.

- **Advantage:** Fast compilation.
- **Disadvantage:** Cannot handle "forward references" (jumping to a label defined later in the code) easily.

Two-Pass Assembler

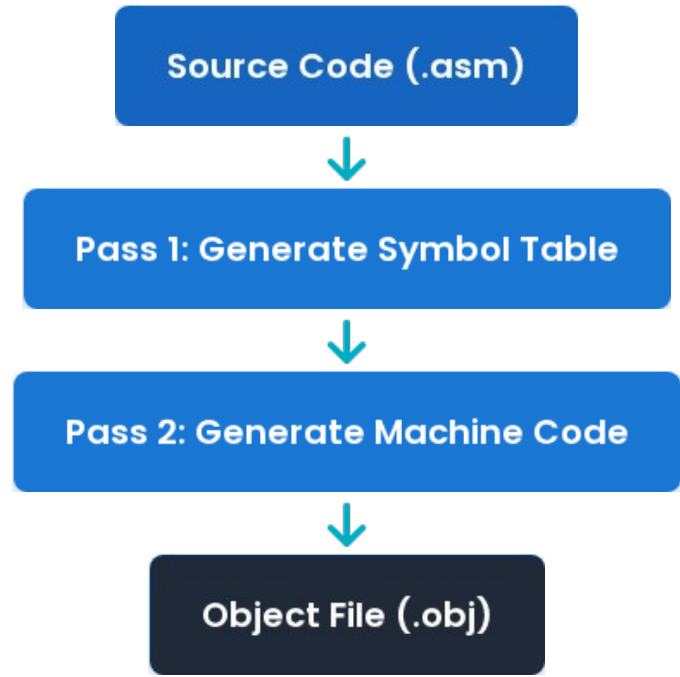
Reads the source code twice to solve the forward reference problem.

- **Pass 1:** Scans the code and builds a *Symbol Table* mapping labels to memory addresses.
- **Pass 2:** Translates the mnemonics using the opcode table and replaces labels with addresses from the Symbol Table.



Assembler

Flowchart of Assembler



Advantages & Applications

Advantages of using an Assembler

- Eliminates the need to memorize addresses
- Allows the use of symbolic names for memory locations, making code easier to modify.
- Catches basic syntax errors before execution.
- Supports macros to reuse code blocks.

Applications

- **MASM (Microsoft Macro Assembler):** Used for Windows programming.
- **NASM (Netwide Assembler):** Popular cross-platform assembler for x86.
- **GNU Assembler (GAS):** Default backend for the GCC compiler.



Assembl
er

Exam Points & Daily Quiz

Important Examination Point: University exams frequently ask to explain the "Forward Reference Problem" and how a Two-Pass Assembler solves it using a Symbol Table. Practice drawing the flowchart.

Daily Quiz

1. What is the output file extension typically generated by an assembler before linking?
2. During which pass does a two-pass assembler construct the Symbol Table?



Instructions

Objective & Introduction

Topic Objective: To comprehend the structure and types of assembly language instructions.

Topic Outcome: Students will write basic data transfer and arithmetic instructions.

Introduction

An assembly instruction is a single command given to the processor. Every instruction directs the CPU to perform one fundamental task, such as moving data, performing math, or jumping to a different part of the program.

Instruction Format

A typical assembly instruction consists of four parts (some are optional depending on the instruction):

```
[Label:] Opcode Operand(s) [; Comment]
```

- **Label (Optional):** A symbolic name representing the memory address of the instruction. Used for cross-referencing.
- **Opcode (Required):** The operation code (mnemonic) specifying the action to perform (e.g., ADD, SUB, MOV).
- **Operands (Varies):** The data or registers the opcode acts upon (e.g., variables, constants, registers).
- **Comment (Optional):** Ignored by the assembler, starts with a semicolon (;) for documentation.



Instructions

Data Transfer: The MOV Instruction

Definition

The MOV (Move) instruction copies data from a source operand to a destination operand. The source

Syntax: MOV destination, source

Rules & Restrictions (x86)

- Both operands must be of the same size (e.g., both 8-bit or both 16-bit).
- Both operands cannot be memory locations simultaneously (cannot move memory to memory directly).
- The instruction pointer (IP/PC) cannot be the destination.



Instructions

Arithmetic: The ADD Instruction

Definition

The ADD instruction adds the source operand to the destination operand and stores the result in the destination operand.

Syntax: ADD destination, source

Logic: destination = destination + source

Examples:

```
MOV AX, 10 ; AX = 10
MOV BX, 20 ; BX = 20
ADD AX, BX ; AX = 10 + 20 = 30 (BX remains 20)
```



Instruction
ns

Arithmetic: The SUB Instruction

Definition

The SUB (Subtract) instruction subtracts the source operand from the destination operand and stores

Syntax: SUB destination, source

Logic: $\text{destination} = \text{destination} - \text{source}$

Important Note on Flags:

Arithmetic instructions (ADD, SUB) modify the CPU's Status Flags (like Zero Flag, Carry Flag, Sign Flag), which are then used by conditional jump instructions to make decisions.

Execution Process & Registers

Common General Purpose Registers (x86 16-bit)

- **AX (Accumulator):** Used for arithmetic and
- **BX (Base):** Used as a base pointer for
- **CX (Counter):** Used for loop counting.
- **DX (Data):** Used for I/O operations and large

These can be split into 8-bit registers (AH, AL, BH, BL, etc.)

$AX = AH | AL$

16-bit register composed of High and Low 8-bit bytes.



Instructions

Assembly Code Example & Flow

Program: Swap two numbers (A and B)

```
; Assume numbers are in AL and BL
MOV AL, 05H ; AL = 05
MOV BL, 09H ; BL = 09

; Swapping logic using CL as temp
MOV CL, AL ; CL = 05
MOV AL, BL ; AL = 09
MOV BL, CL ; BL = 05

HLT ; Stop Execution
```

Register States

Initial: AL=05, BL=09, CL=??

Step 1: AL=05, BL=09, CL=**05**

Step 2: AL=**09**, BL=09, CL=05

Step 3: AL=09, BL=**05**, CL=05

Swapped successfully!



Instructions

Machine Code Relationship

How does "MOV AL, 05H" turn into binary?

Assembly Instruction	Opcode (Hex)	Operand (Hex)	Machine Code (Binary)
MOV AL, 05H	B0	05	10110000 00000101
ADD AL, BL	00	D8	00000000 11011000
HLT	F4	-	11110100

Note: Hex opcodes vary based on specific processor architecture manual.



Instructions

Practical Example

Write an assembly snippet to compute: $Y = A + B - C$

Assume $A=10$, $B=20$, $C=5$. We will store result Y in register DX .

```
MOV AX, 10 ; Load A into AX
MOV BX, 20 ; Load B into BX
MOV CX, 5  ; Load C into CX

ADD AX, BX ; AX = AX + BX (AX is now 30)
SUB AX, CX ; AX = AX - CX (AX is now 25)

MOV DX, AX ; Store final result in DX (DX = 25)
HLT ; End program
```



Instructions

Exam Points & Daily Quiz

Important Examination Point: Always write comments beside your assembly code in exams. Evaluators look at comments to understand your logic, even if the exact syntax has minor flaws.

Daily Quiz

1. Which register is traditionally known as the Accumulator?
2. True or False: You can move data directly from one memory location to another using a single instruction.
3. Write the MOV instruction to subtract 86 from AX.



Assembly VS Machine Lang

Objective & Introduction

Topic Objective: To distinguish between symbolic low-level code and pure binary hardware code.

Topic Outcome: Students will clearly articulate the differences and the necessity of both languages.

Introduction

While often discussed together, Assembly and Machine language are not the same. One is meant for human programmers to read and write, while the other is the only thing the silicon processor physically responds to via electrical signals.



Assembly VS Machine Lang

Detailed Comparison Theory

Machine Language (First Generation)

Consists entirely of strings of 0s and 1s. This is the lowest possible level. It requires no translation. It is highly prone to human error—a single flipped bit changes the entire instruction.

Assembly Language (Second Generation)

Uses alphanumeric mnemonics. It provides a human-readable abstraction over machine code. While easier than binary, it still requires the programmer to manage CPU registers and memory manually.



Assembly VS Machine Lang

Professional Comparison Table

Feature	Assembly Language	Machine Language
Format	Mnemonics (English-like abbreviations)	Binary format (0s and 1s)
Readability	Difficult, but manageable for humans	Nearly impossible for humans to read
Translation	Requires an Assembler to execute	Executed directly by the CPU; no translation
Modification	Easier to modify and debug	Extremely difficult to modify
Hardware Dependency	Dependent (architecture specific)	Dependent (architecture specific)



Assembly VS Machine Lang

Translation Process

The transformation from symbolic code to executable signals.

ADD AX, BX

Human writes
mnemonic.



Assem
bler
looks
up
Opcode

00000000 11011000

CPU receives voltage
highs/lows.

The Assembler maintains a hardcoded lookup table mapping every mnemonic to its binary equivalent.



Assembly VS Machine Lang

Exam Points & Daily Quiz

Important Examination Point: Be prepared to draw the comparison table. Emphasize that BOTH languages are machine-dependent, unlike C or Java.

Daily Quiz

1. Which generation of programming languages does Machine Code belong to?
2. Does Machine Language require a compiler or assembler? Why or why not?



Pipelining

Objective & Introduction

Topic Objective: To understand how modern processors achieve high throughput by overlapping

Topic Outcome: Students will calculate speedup using pipeline timing diagrams and identify basic pipeline hazards.

Introduction

In older processors, an instruction had to finish completely before the next one could begin. This left parts of the CPU idle. Pipelining is an implementation technique where multiple instructions are overlapped in execution, vastly improving CPU throughput.

Definition & Need

Definition

Pipelining is the process of accumulating instruction from the processor through a pipeline. It allows storing and executing instructions in an orderly process, akin to a factory assembly line.

Need for Pipelining

- **Throughput:** Increases the number of instructions completed per unit time.
- **Resource Utilization:** Keeps all CPU components (ALU, Memory unit, Registers) busy simultaneously.
- **Clock Speed:** Shorter data paths in pipeline stages allow for higher processor clock frequencies.



Pipelining

Concept & ILP

Instruction Level Parallelism (ILP)

Pipelining is a form of ILP. It relies on the fact that an instruction cycle is composed of several independent steps. While the ALU executes Instruction 1, the memory unit can fetch Instruction 2.

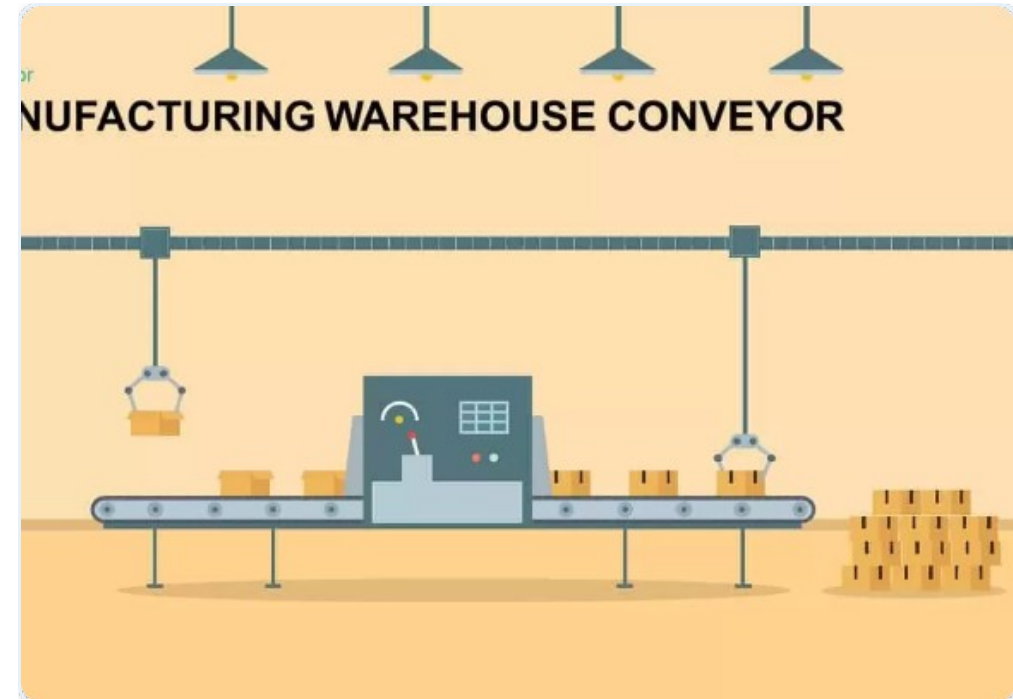
Non-Pipelined vs Pipelined

Non-Pipelined: Laundry: Wash (30m) -> Dry (40m) -> Fold (20m). Total for 1 load = 90m. Total

Pipelined: 360m as Load 1 goes in the dryer, Load 2 goes in the washer. Total for 4 loads is much less than 360m.

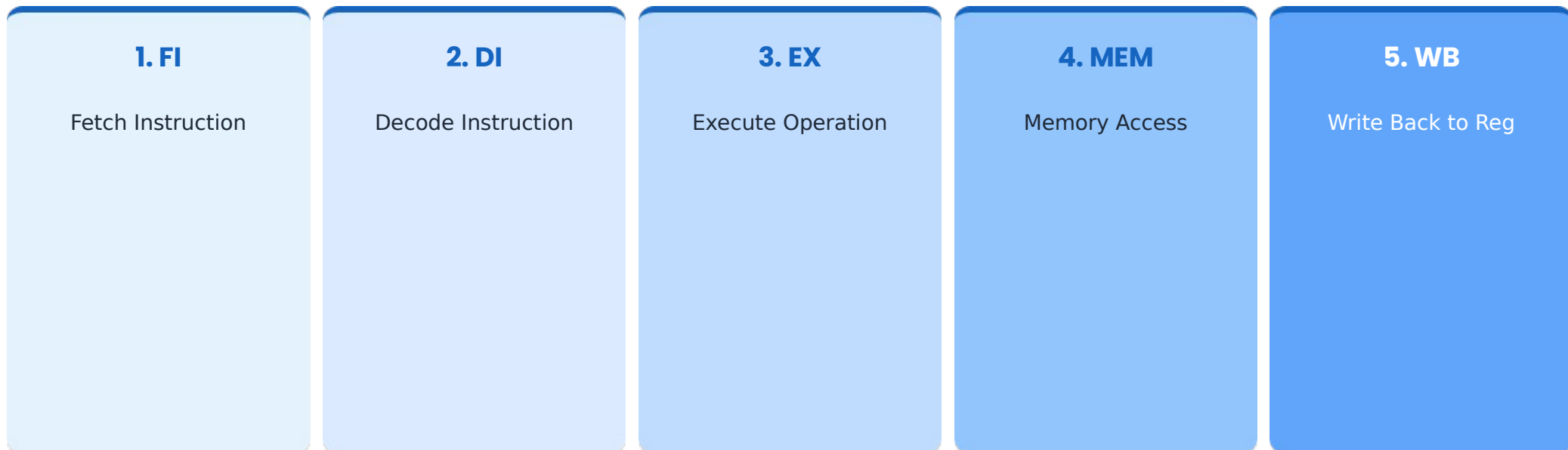
Real-Life Analogy Diagram

Just like a car factory assembly line, multiple cars are built simultaneously. Station 1 installs the engine while Station 2 paints another car. The total time to build one car doesn't change, but the number of cars produced per day skyrockets.



Pipeline Architecture: Classic 5-Stage

The classic RISC pipeline divides instruction execution into 5 discrete stages:



Detailed Pipeline Stages

- **Fetch (FI):** Read the instruction from memory using the Program Counter (PC).
- **Decode (DI):** Interpret the opcode and read operand values from the register file.
- **Execute (EX):** The ALU performs the arithmetic or logic operation. For memory instructions, it
- **Memory Access (MEM):** If it's a Load or Store, read from or write to the data memory. Otherwise,
- **Write Back (WB):** Write the final result back into the destination register.



Pipeline Timing Diagram

Visualizing the overlap across clock cycles.

Instruction \ Clock Cycle	1	2	3	4	5	6	7	8
Instr 1	FI	DI	EX	MEM	WB			
Instr 2		FI	DI	EX	MEM	WB		
Instr 3			FI	DI	EX	MEM	WB	
Instr 4				FI	DI	EX	MEM	WB

Notice: At Clock Cycle 5, all 5 stages of the CPU are active simultaneously!

Speed Improvement Formula

How much faster is a pipelined processor?

$$\text{Speedup (S)} = \frac{\text{Time}_{\text{Non-Pipelined}}}{\text{Time}_{\text{Pipelined}}}$$

$$S = \frac{n \times k \times t_p}{(k + n - 1) \times t_p}$$

Where **n** = number of instructions, and **k** = number of pipeline stages.

Ideal Speedup approaches 'k' for a very large number of instructions.

Advantages & Disadvantages

Advantages

- Vast increase in CPU throughput.
- Allows for higher clock rates since there is less logic between registers.
- More efficient use of silicon resources on the chip.

Disadvantages

- Complexity of CPU design increases
- Instructional latency (time for one instruction) slightly increases due to stage registers.
- Vulnerable to Pipeline Hazards.



Introduction to Pipeline Hazards

A **Hazard** is a situation that prevents the next instruction in the instruction stream from executing during its designated clock cycle. Hazards reduce the performance from the ideal speedup.

When a hazard occurs, the pipeline must **STALL** (insert a "bubble" or empty cycle) to resolve the conflict, wasting clock cycles.

Types of Hazards

1. Structural Hazard

Occurs when hardware cannot support all overlapped instructions simultaneously (e.g., two instructions want to use memory at the same time).

2. Data Hazard

Occurs when an instruction depends on the result of a previous instruction that is still in the pipeline (Read-After-Write).

3. Control Hazard

Also known as Branch Hazards. Occurs when the pipeline makes the wrong decision on a branch prediction (e.g., IF/ELSE statements).



Pipeline g

Data Hazard Example

```
Instr 1: ADD AX, BX ; Calculates new AX  
Instr 2: SUB CX, AX ; Needs the new AX to calculate CX
```

If Instr 2 tries to read AX during its Decode stage, but Instr 1 hasn't reached its Write Back stage yet, Instr 2 will read the **OLD, WRONG** value of AX.

Solution: Forwarding (bypassing) or inserting stalls (NOPs).



Pipeline g

Exam Points & Weekly Assignment

Daily Quiz

1. What is the ideal speedup of a 6-stage pipeline?
2. Name the hazard caused by a lack of hardware resources.

Weekly Assignment: Draw the timing diagram for 4 instructions in a 5-stage pipeline where Instruction 3 has a Data Hazard dependent on Instruction 2, causing a 2-cycle stall.





Course Review

Complete Course Outcome 5 Review

We have reached the end of the syllabus. Let's step back and look at the entire computer system from the bottom up.

Complete Course Outcome 5 Playlist: <https://www.youtube.com/watch?v=L9X7XXfHYdU&list=PLxCzCOWd7aiHMonh3G6QNKq53C6oNXGrX>

Review: Digital Logic (Unit 1)

The Foundation

- **Logic Gates:** AND, OR, NOT, NAND, NOR, XOR. The building blocks of silicon.
- **Combinational Circuits:** Multiplexers (Data selectors), Decoders, Adders. Output depends only on current input.
- **Sequential Circuits:** Flip-Flops, Registers, Counters. Have memory; output depends on current input and past state.

Memory Trick:

Combinational = **C**urrent input only.

Sequential = **S**tored memory matters.



Course Review

Review: CPU & Registers (Unit 2)

Central Processing Unit

The brain of the computer, composed of the ALU and the Control Unit.

- **ALU (Arithmetic Logic Unit):** Does the actual math and logic. Built from Adders and Logic Gates
- **Control Unit:** The conductor. Fetches instructions, decodes them, and sends control signals. Can be Hardwired (fast, complex) or Microprogrammed (flexible, slower).
- **Registers:** PC (Next instruction address), IR (Current instruction code), MAR (Memory Address Register), MDR (Memory Data Register).



Course Review

Review: Memory Organization (Unit 3)

Memory Hierarchy

Faster memory is expensive and small. Slower memory is cheap and large.

- Registers (Fastest, Smallest)
- Cache Memory (L1, L2, L3)
- Main Memory (RAM)
- Secondary Storage (SSD/HDD - Slowest, Largest)

Cache Mapping

- Direct Mapping
- Associative Mapping
- Set-Associative Mapping

Review: Input/Output (Unit 4)

How the CPU talks to the outside world.

Programmed I/O

CPU constantly polls the device. Wastes a lot of CPU time.

Interrupt-Driven I/O

Device sends an interrupt signal to the CPU when ready.
Better CPU utilization.

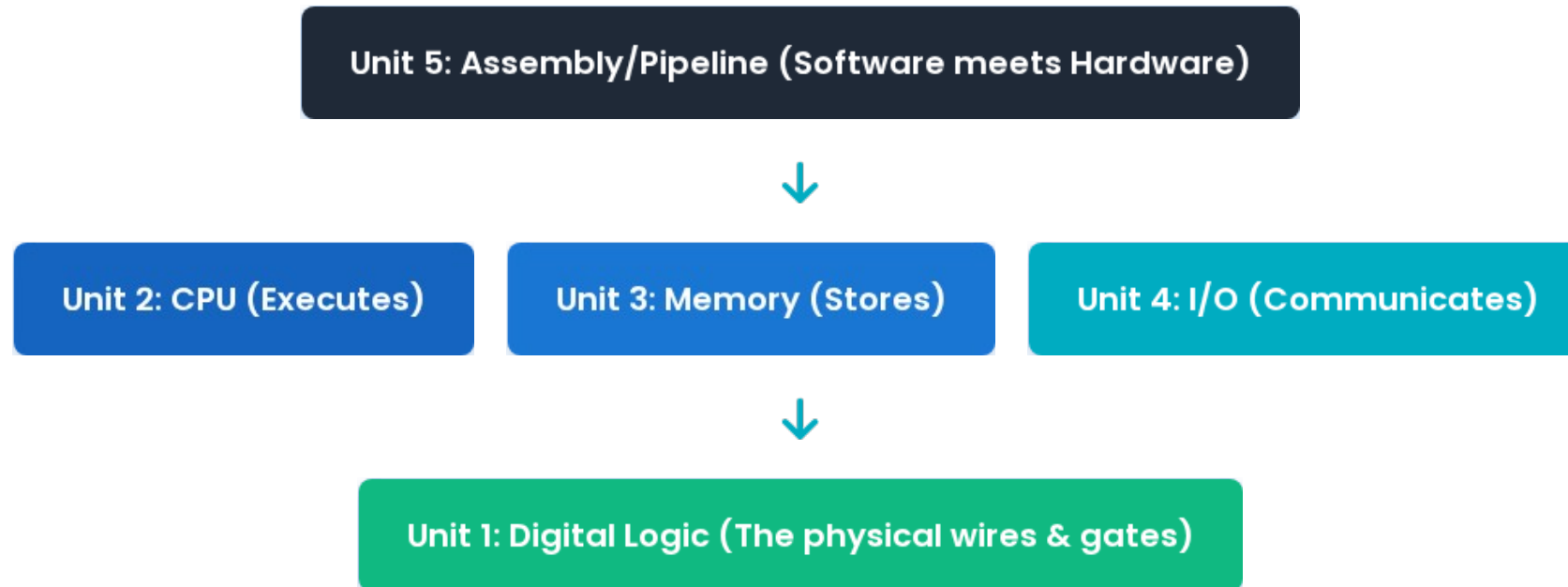
DMA (Direct Memory Access)

DMA controller handles block data transfers directly to memory, bypassing the CPU entirely.



Course Review

Connecting All Units (Concept Map)





Course Review

Summary Tables & Revision

Component	Speed	Size	Unit Discussed
Registers	< 1ns	Bytes	Unit 2 & 5
Cache (L1)	~ 1-2ns	Kilobytes	Unit 3
RAM	~ 10-50ns	Gigabytes	Unit 3
Hard Drive	Millions of ns	Terabytes	Unit 4

Understanding this table is key to understanding why we need Registers for Assembly (Unit 5).



Course Review

Important Formula Revision

Cache Hit Ratio

$$H = \frac{\text{Hits}}{\text{Hits} + \text{Misses}}$$

Pipeline Speedup

$$S = \frac{n \times k}{k + n - 1}$$

Average Memory Access Time (AMAT)

$$\text{AMAT} = \text{HitTime} + (\text{MissRate} \times \text{MissPenalty})$$



Course Review

Important Definitions Checklist

- **Instruction Set Architecture (ISA):** The part of the computer architecture related to
- **Von Neumann Architecture:** Both instructions and data are stored in the same memory.
- **Harvard Architecture:** Separate memory space for data and instructions.
- **Endianness:** Order of bytes in computer memory (Big-Endian vs Little-Endian).
- **Micro-operation:** The most basic, atomic operation performed by a CPU on data in registers.

Frequently Asked Questions

Q: RISC vs CISC?

RISC has simple, single-cycle instructions (highly pipelinable). **CISC** has complex instructions that may take multiple cycles.

Q: Hardwired vs Microprogrammed?

Hardwired is purely physical gates (fast).

Microprogrammed uses a small control memory (flexible, easier to design).

Q: Why use Cache?

To bridge the massive speed gap between the ultra-fast CPU registers and the relatively slow Main Memory (RAM) using the Principle of Locality.



Unit MCQs

Test Your Knowledge

1. An Assembler translates:

- A) High-level to Machine
- B) Assembly to Machine
- C) Machine to Assembly

2. In a 5-stage pipeline, 'MEM' stands for:

- A) Memory Access
- B) Memory Erase
- C) Mnemonic Execute

3. Branch instructions typically cause which hazard?

- A) Data Hazard
- B) Structural Hazard
- C) Control Hazard

4. Which register holds the address of the next instruction?

- A) Instruction Register (IR)
- B) Accumulator (AX)
- C) Program Counter (PC)

Answer Key: 1-B, 2-A, 3-C, 4-C



Glossar y

Key Terminology

- **Opcode:** Operation Code. The part of the instruction specifying what to do.
- **Operand:** The data or memory address upon which the opcode operates.
- **Mnemonic:** A short, human-readable abbreviation for an assembly instruction (e.g., ADD, SUB).
- **Pipeline Bubble:** A stall cycle inserted into the pipeline to resolve a hazard.
- **Throughput:** The amount of work (instructions) completed in a given amount of time.
- **Symbol Table:** Data structure created by the assembler in Pass 1 to track label addresses.



Preparatio
n

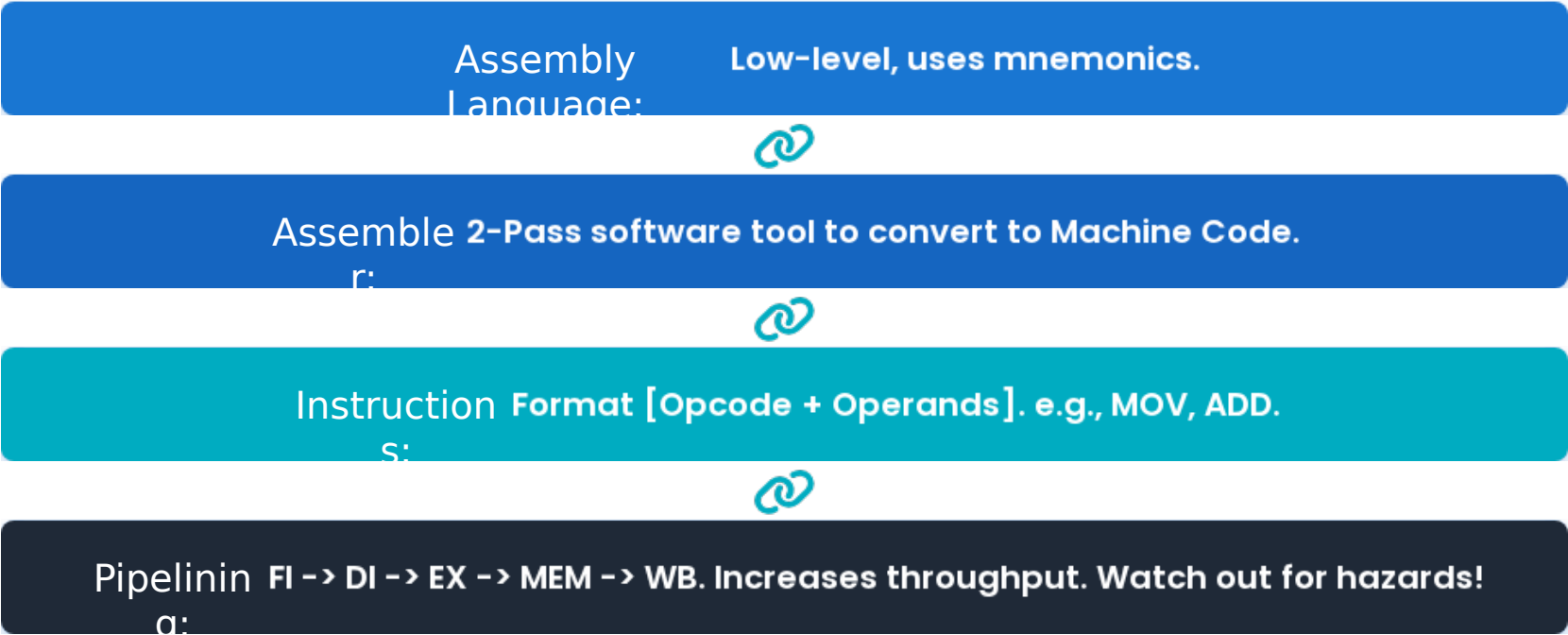
Expected Long Questions (Unit 5)

1. Explain the working of a Two-Pass Assembler with a neat flowchart. (13 Marks)
2. Differentiate between Assembly Language and Machine Language in detail. (8 Marks)
3. Explain the 5 stages of a classic RISC pipeline. Draw the timing diagram for 5 instructions. (13 Marks)
4. What are Pipeline Hazards? Classify and explain them with examples. (13 Marks)
5. Write an assembly language program to add two 8-bit numbers. (8 Marks)



Recap

One Minute Revision Mind Map





Thank You!

Questions &
Discussion

Slide 73 / 73